Willkommen



LS 2 –
Entwicklung einer Web-App
für das Fantasy-Spiel

Pattern Wars 2.5

Situationsbeschreibung

Ihr seid Teil eines Entwicklerteams, das für das fiktive Fantasy-Game "Pattern Wars 2.5" eine Web-App erstellen soll. Ziel ist es, eine interaktive Anwendung zu entwickeln, die die verschiedenen Charaktere des Spiels präsentiert und den Nutzern die Möglichkeit bietet, sich anzumelden, Charaktere zu erstellen und Details zu erkunden.

Die Web-App soll mit **Python und Flask** entwickelt werden und grundlegende Webtechnologien wie **HTML, CSS** und **Bootstrap** nutzen. Ihr werdet verschiedene Funktionen implementieren, darunter eine **Login-Seite**, eine **Übersicht über die Charaktere** sowie ein **Formular zur Charaktererstellung**. Dabei liegt der Fokus auf dem Verständnis von **HTTP-Methoden**, der **REST-Architektur** und der Nutzung von **Templates**.

Bearbeitet die Aufgaben in kleinen Teams und nutzt euren kreativen Spielraum, um das Projekt ansprechend zu gestalten. Ziel ist es, sowohl die technischen als auch die gestalterischen Fähigkeiten weiterzuentwickeln und die Zusammenarbeit im Team zu fördern.

Was erwartet euch...

A

Flask
REST
HTTP

B

Rendering
Formulare
Bootstrap

C

Jinja2
User
Sessions

D

Datenbank
anbindung

HTTP | REST | Flask

Eine kurze Einführung in Grundkonzepte



Hypertext Transfer Protocol (HTTP)

- Kommunikationsprotokoll im World Wide Web
- Grundlage für Datentransfer zwischen Clients und Servern
- Zustandslos: Jede Anfrage ist unabhängig von den vorherigen



HTTP-Requests und -Responses

- **Methoden**: GET, POST, PUT, DELETE
- **URL**: Identifiziert Ressource
- **Header**: Metainformationen
- **Body**: Optional, z. B. Daten bei POST



HTTP-Methoden im Detail

- **GET**: Daten abrufen
- **POST**: Neue Daten senden
- **PUT**: Daten aktualisieren
- **DELETE**: Daten löschen
- **Status Codes**:

```
200 OK, 404 Not Found, 500 Internal Server Error
```



Representational State Transfer (REST)

- Architekturstil für Web-APIs
- Jede Ressource wird über eine eindeutige URL angesprochen
- Nutzung der HTTP-Methoden zur Manipulation von Ressourcen
- Jede Ressource hat eine eindeutige URL

`GET /users` (Laden von Nutzern) | `POST /users` (Neuen Nutzer anlegen)



Prinzipien von RESTful APIs

- **Zustandslosigkeit**:

  Keine Speicherung des Zustands auf dem Server

- **Cookies und Caching**:

  Wiederkehrende Informationen und Optimierung der Effizienz

- **JSON** als gängigstes Format zur Übertragung von Daten



Vorteile von REST

- **Einfachheit**: Nutzung der bekannten HTTP-Methoden
- **Flexibilität**: Einfacher Zugriff durch eindeutige URLs
- **Skalierbarkeit**: Zustandslosigkeit erleichtert das Skalieren



Was ist FLASK?

- Ein leichtgewichtiges Python-Webframework
- Geeignet für kleine bis mittelgroße Webanwendungen
- Schnell zu erlernen, ideal für REST-APIs



Vorteile von FLASK

- **Leichtgewichtig**:

  Keine unnötigen Abstraktionen, einfach zu verwenden

- **Flexibilität**:

  Kann für einfache Webseiten bis hin zu komplexen Web-APIs genutzt werden

- **Erweiterbarkeit**:

  Unterstützt Erweiterungen, z. B. für Authentifizierung , Datenbankanbindung, …



Grundaufbau

einer Flask Web-App



„Hello World“ als Web-App



flask-Bibliothek einbinden

aktuelle Datei als Hauptdatei festlegen

Decorator mit Route (URL) / Ressource

Plaintext, kein HTML (!)

```python
from flask import Flask

# Initialisieren der Flask-Anwendung
app = Flask(__name__)

# Definieren der ersten Route
@app.route('/')
def home():
    return "Hello World"

# Starten der Anwendung im Debugging-Modus
if __name__ == "__main__":
    app.run(debug=True)
```

Parametrisierung von Routen



```python
# **Personalisierte Begrüßung mit Parametern**
# Übergabe des Nutzernamens über eine parametrisierte URL

@app.route('/welcome/<name>')
def welcome(name):
    return f"Willkommen, {name}!"
```

Individualisierung der URL

muss im Parameter übergeben werden

GET-Requests in Flask

in der URL:
`/create_user?user_id=1&username=Tobi`



```python
@app.route('/create_user', methods=['GET'])
def create_user():
    user_id = request.args.get('user_id')
    username = request.args.get('username')
    return f"Benutzer {username} mit ID {user_id} wurde erfolgreich erstellt!"
```

erlaubte HTTP-Methode

Lesen von URL-Argumenten
→ Query-Paramter

POST-Requests in Flask

im body:

```
{
"user_id": 1,
"username": "Tobi"
}
```



```python
@app.route('/create_user', methods=['POST'])
def create_user():
    # Benutzerinformationen aus dem Request-Body abrufen
    user_data = request.get_json()
    user_id = user_data.get('user_id')
    username = user_data.get('username')

    return f"Benutzer {username} mit ID {user_id} wurde erfolgreich erstellt!"
```

erlaubte HTTP-Methode

Lesen der JSON-Daten

→ testen z.B mit Postman

Aufgaben

1. Überlegt euch sinnvolle Routen für die Web-App zu „Pattern Wars 2.5“!
2. Erstellt Dekoratoren mit entsprechenden Methoden in eurem Projektumfeld.
3. Macht euch grundlegende Gedanken zum Aussehen eurer Web-App.
   Entwickelt dazu erste Sketche für Layout und Design.



Websites in Flask

Formulare & Rendering von HTML-Dateien



Aufbau eines HTML-Dokuments

Datei: `index.html`

Standard: HTML5

Sprache (de, en, fr, ...)

Metainformationen

responsives Design

```
<!DOCTYPE html>
<html lang="de">

  <head>

    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title> Website | Unterseite </title>

  </head>

  <body>
    <h1> Herzliche Willkommen auf der Website der ITO2! </h1>
    <p> Zurücklehnen und genießen... </p>
  </body>

</html>
```

öffnendes Tag



schließendes Tag

Sichtbarer Website-Bereich
(hier: mit großer Überschrift und Text-Paragraph)

Rendern von HTML-Dateien in Flask



Bibliothek `render_template` importieren

```
1  from flask import Flask, request, render_template
2
3  app = Flask(__name__)
4
5  @app.route('/')
6  def index():
7      return render_template("index.html")
```

Templating-Methode mit HTML-Datei
als Parameter returnen

Einfache HTML-formulare

Einfaches Eingabefeld
(`type: text, password, ...`)

Attribut `action`:
Weiterleitung an angegebene Route

```
<h1> Bitte geben Sie Ihren Namen ein! </h1>
<br>
<form action="/greet">
    <input type="text" name="name">
    <button type="submit">Klick mich! </button>
</form>
```

```
9  @app.route("/greet")
10 def greet():
11     name = request.args.get("name")
12     return render_template("greet.html", name=name)
```

```
<h1> Herzlich willkommen, {{ name }}!</h1>
```

Wichtig!
Attribut `name`
Weiterleitung der Eingabe als Variable für Flask/Jinja2

Aufgaben

1. Ladet alle Vorlagendateien in euer pythonanywhere-Projekt.
2. Erweitert die Datei index.html um eine Login-Maske mit Username und Passwort, die auf die Seite login.html weiterleitet und die übermittelten Daten anzeigt.

   → *Nutzt einmal GET und einmal POST, um die Unterschiede zu sehen!*



Templating & Python

mit der Engine Jinja2



Base-Template



```
1  <!DOCTYPE html>
2  <html lang="de">
3
4    <head>
5
6      <meta charset="utf-8">
7      <meta name="viewport" content="width=device-width, initial-scale=1.0">
8      <title> Website - {% block title %} {% endblock %} </title>
9
10   </head>
11
12   <body>
13
14       {% block content %}
15
16       {% endblock %}
17
18   </body>
19
20 </html>
```

Veränderliche Blöcke/
Vorlagen

Jinja2-Notation für das
Einfügen von Python-Code
{ % % }

Erben & Erweitern eines Template

`extends`-Schlüsselwort + Master-Template

Wichtig!
alle `html`-Dateien im Ordner `/templates`



```
1  {% extends "base.html" %}
2
3      {% block title %} ITO2 - Registrierung {% endblock %}
4
5      {% block content %}
6
7          <h1 class="mt-3"> Benutzerregistrierung </h1>
8
9          <form method="post">
10             <input type="text" name="username" required>
11             <input type="password" name="password" required>
12             <button type="submit"> Klick mich! </button>
13         </form>
14
15     {% endblock %}
```

sorgt für hiding des Passwortes

einfache, Client-seitige Überprüfung

Flask-Module redirect & url_for

Import der Module

Globale Variable (lokales JSON zum Testen einer User-Datenbank)

```
1  from flask import Flask, request, render_template, url_for, redirect
2
3  app = Flask(__name__)
4
5  users = [{"username": "test", "password": 123456789}]
6
7  @app.route('/', methods=['GET', 'POST'])
8  def index():
9      if request.method == 'POST':
10         username = request.form.get("username")
11         password = request.form.get("password")
12         users.append({"username": username, "password": password})
13         return redirect(url_for('greet', username=username))
14     return render_template("index.html")
```

Auswahl von Aktionen je nach request-Methode



Weiterleitung an übergebene Route

Erzeugung der Route aus Name der Python-Funktion

Schleifen & Verzweigungen

Globale user-Liste (JSON) wird an Route übergeben

```python
@app.route('/listUsers')
def listUsers():
    return render_template("listUsers.html", users=users)
```



```
{% extends "base.html" %}

    {% block title %} ITO2 - Userliste {% endblock %}

    {% block content %}

        <div class="container-fluid">

            {% for user in users %}
            <div class="row border">
                <div class="col"> {{ user.username }} </div> <div class="col"> {{ user.password }} </div>
            </div>
            {% endfor %}

        </div>

    {% endblock %}
```

for-each-Schleife in Jinja2-Notation

Zugriff auf keys aus JSON-Datei

Sessions in flask – Teil 1

Import des session-Moduls

```
1  from flask import Flask, request, render_template, url_for, redirect, session
2
3  app = Flask(__name__)
4  app.secret_key = "supersecretkey"
```

App-Eigenschaft secretkey setzen



```
10     if request.method == 'POST':
11         username = request.form.get("username")
12         password = request.form.get("password")
13         users.append({"username": username, "password": password})
14         session["username"] = username
15         session.modified = True
16         return redirect(url_for('greet', username=username))
```

nach Login:
Username in Session-Objekt speichern

Sessions in flask – Teil II

Überprüfung, ob aktuell ein User im Session-Objekt

Bsp
Einblenden v. Buttons abhängig v. Anmeldung

```
{% if session.get("username") %}
    <form action="{{ url_for('logout') }}" method="post">
        <button type="submit"> Logout ({{ session.get('username') }}) </button>
    </form>
{% endif %}
```

Zugriff auf aktuell angemeldeten Nutzer(namen)



```
28  @app.route('/logout', methods=['POST'])
29  def logout():
30      session.pop("username", None)
31      return redirect(url_for('index'))
```

wichtig!
Route ohne zugehörige HTML-Seite

Löschen d. aktuellen Session-Objektes über pop-Befehl